ML LAB 14

NAME: Bojja Rakshitha

SECTION:C

SRN:PES2UG23CS134

1. Introduction

The objective of this laboratory exercise was to design, build, and train a Convolutional Neural Network (CNN) using the PyTorch framework[1]. This model was specifically tasked with accurately classifying images of various hand gestures into one of three predefined categories: **'rock'**, **'paper'**, or **'scissors'**. The implementation involved completing a boilerplate notebook, setting up the data pipeline, training the model on the "Rock Paper Scissors" dataset, and evaluating its performance to fulfill the requirements of the lab[2].

2. Model Architecture

The custom CNN architecture, named RPS_CNN, was designed with a sequential block structure comprising three convolutional layers followed by a fully-connected classifier[3].

Convolutional Block Description

The feature extraction component consists of three stacked blocks, each implementing the sequence: **Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool2d**[4]. This design

progressively extracts features while reducing the spatial dimensions.

- **Layer 1:** Takes the 3-channel RGB image input and outputs **16 channels**. It uses a **kernel size of 3×3** with padding=1[5]. It is followed by **Max Pooling with a kernel size of 2**[6].
- **Layer 2:** Increases the depth from 16 to **32 channels**, also using a **kernel size of 3×3** with padding=1[7]. It is followed by Max Pooling (kernel size 2)[8].
- **Layer 3:** Further increases the depth from 32 to **64 channels**, using a **kernel size of 3×3** with padding=1[9]. It is again followed by Max Pooling (kernel size 2)[10].

Fully-Connected Classifier Description

The classifier takes the output of the convolutional layers to perform the final class prediction[11].

- The three Max Pooling layers reduce the initial 128×128 image size to 16×16[12]. When flattened, the input size to the classifier is $64 \times 16 \times 16 = \mathbf{16,384}$[13].
- The classifier sequence is: **Flatten** $\rightarrow$ **Linear Layer** (16,384 $\rightarrow$ 256) $\rightarrow$ **ReLU** $\rightarrow$ **Dropout** ($p=0.3$) $\rightarrow$ **Linear Layer** (256 $\rightarrow$ 3)[14].
- The final output layer has 3 neurons, corresponding to the three classes: 'paper', 'rock', and 'scissors'[15].

3. Training and Performance

Key Hyperparameters

The model was trained for a total of **10 epochs** [16]using the following hyperparameters[17]:

- **Optimizer:** Adam [18]
- **Loss Function (Criterion):** nn.CrossEntropyLoss [19]
- **Learning Rate ($\mathbf{lr}$):** 0.001 [20]
- **Batch Size:** 32 [2121]
- **Device:** CPU (since a CUDA-enabled GPU was not available)[22].

Final Test Accuracy

The model was evaluated on the unseen test dataset (438 images)[232323].

Metric	Value
Final Test Accuracy	**96.80%** [24]

4. Conclusion and Analysis

Discussion of Results

The model demonstrated **excellent performance**, achieving a final test accuracy of **96.80%**[25]. This high level of accuracy indicates that the chosen CNN architecture was successful in learning the complex spatial features that distinguish the

three hand gestures (rock, paper, scissors) within the dataset[26]. The low training losses across 10 epochs (ranging from 0.6251 down to 0.0261) suggest the model converged effectively[27].

Challenges Faced

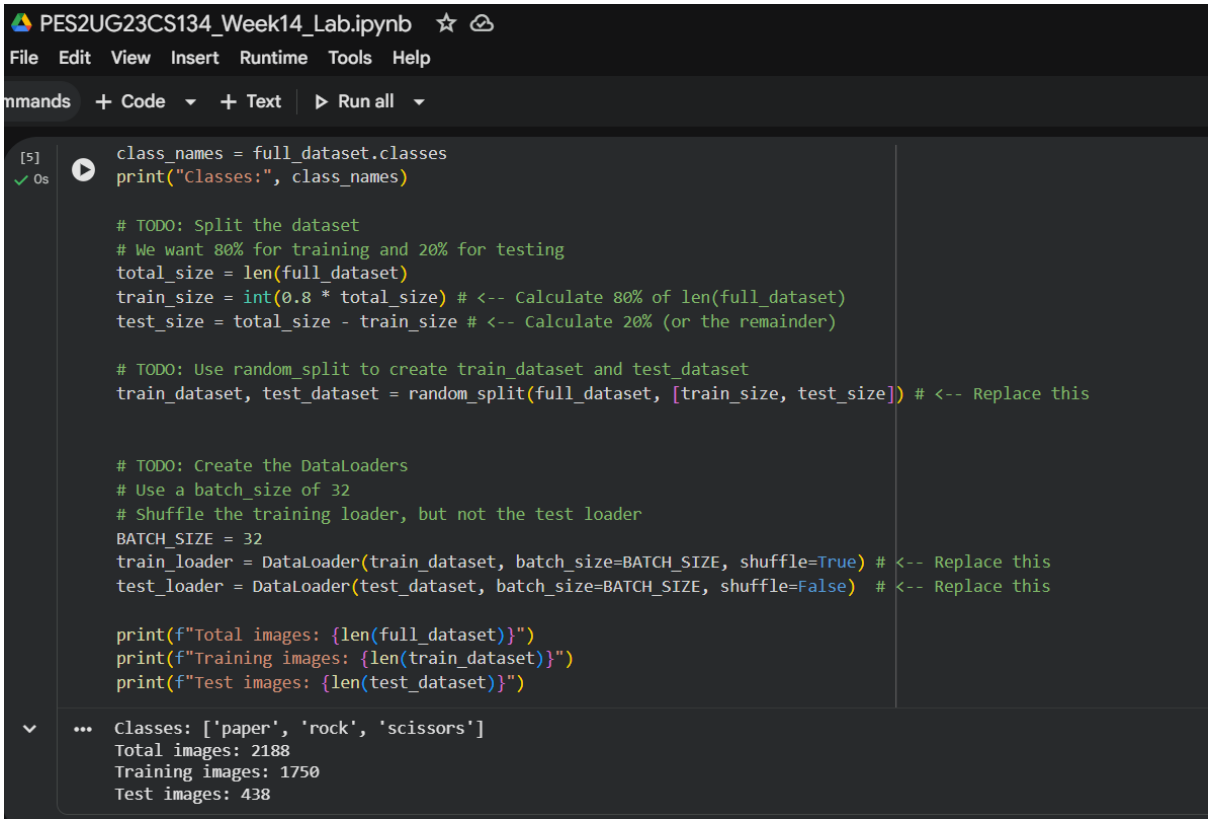
One practical challenge was the potential need for **Kaggle API authentication** to successfully download the dataset outside of a native Kaggle environment, although the provided notebook handled the file movement after download[28]. From a machine learning perspective, high accuracy with a deep network could raise concerns about **overfitting** to the training data, though the use of Dropout was a measure taken to counteract this[29].

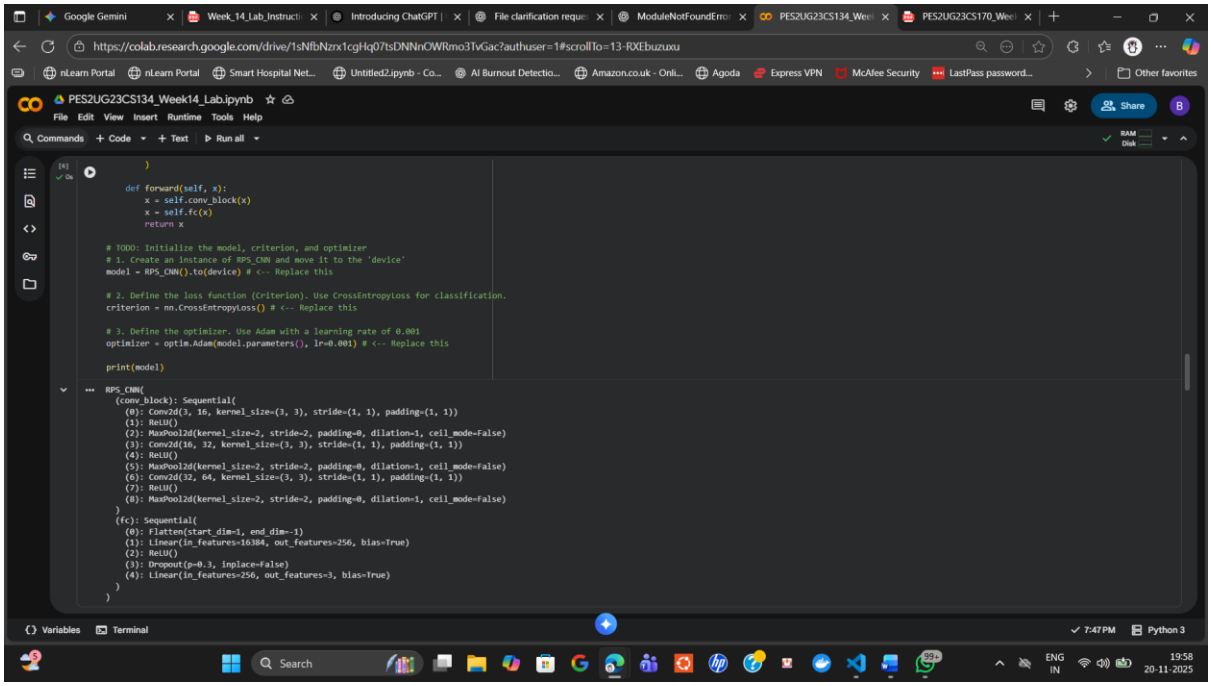
Suggested Improvements

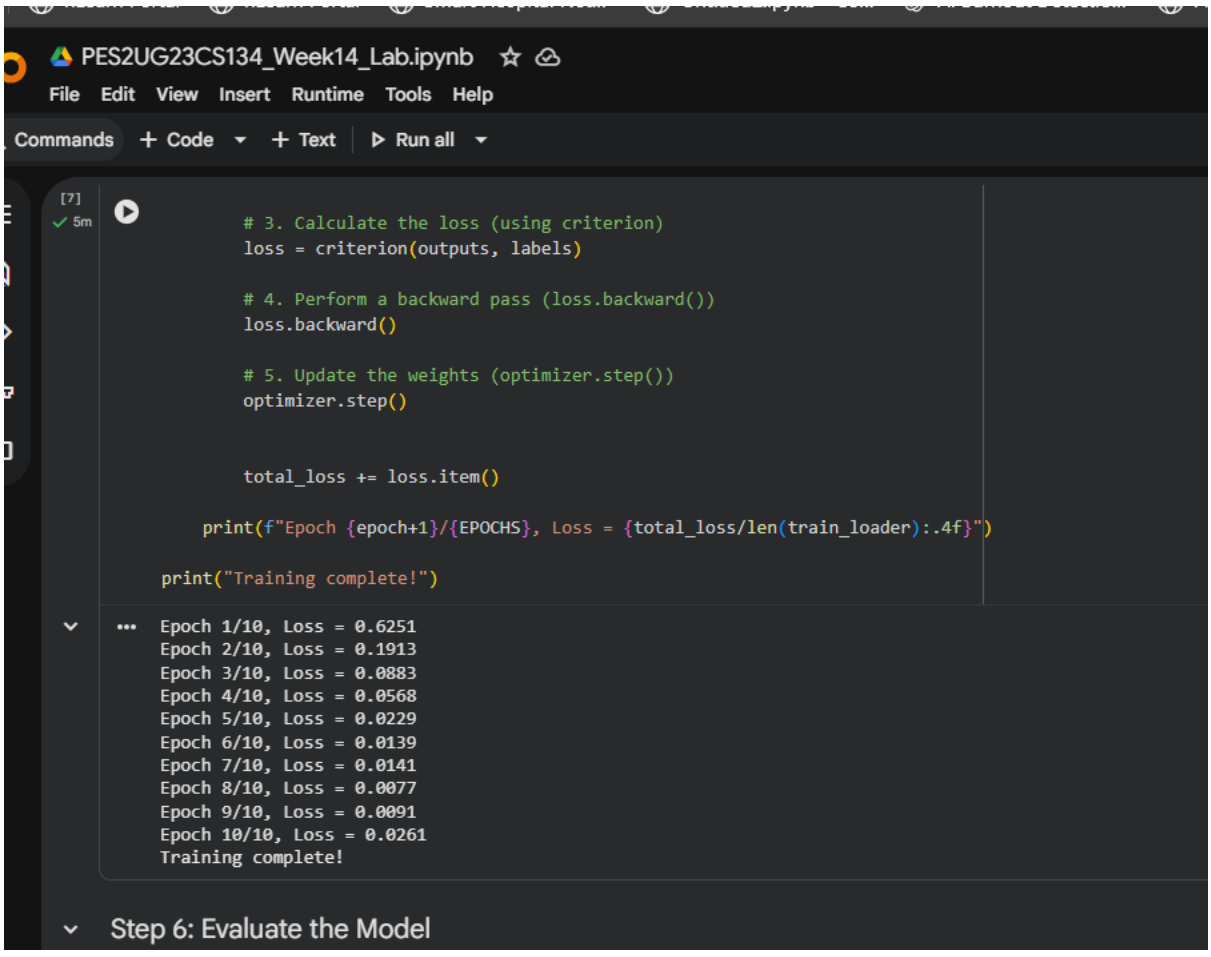
To potentially improve the model's accuracy and generalization capabilities further, the following strategies could be implemented[30]:

1. **Introduce Extensive Data Augmentation:** Implement more aggressive preprocessing techniques (beyond resizing and normalization) such as random rotation, horizontal/vertical flipping, or color jittering. This would artificially increase the diversity of the training set and make the model more robust to variations in real-world lighting or camera angles.
2. **Transfer Learning with Pre-trained Models:** Instead of training the custom architecture from scratch, replace it with a pre-trained, state-of-the-art model (e.g., ResNet

or VGG) and fine-tune its final layers on the Rock Paper Scissors dataset. This approach leverages features learned from millions of general images and is often the most effective way to achieve marginal accuracy gains on specialized image tasks.




PES2UG23CS134_Week14_Lab.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
# TODO: Initialize the model, criterion, and optimizer
# 1. Create an instance of RPS_CNN and move it to the 'device'
model = RPS_CNN().to(device) # <-- Replace this
# 2. Define the loss function (Criterion). Use CrossEntropyLoss for classification.
criterion = nn.CrossEntropyLoss() # <-- Replace this
# 3. Define the optimizer. Use Adam with a learning rate of 0.001
optimizer = optim.Adam(model.parameters(), lr=0.001) # <-- Replace this
print(model)
RPS_CNN(
(conv_block): Sequential(
(0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(1): ReLU()
(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(4): ReLU()
(5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(6): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(7): ReLU()
(8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(fc): Sequential(
(0): Flatten(start_dim=1, end_dim=-1)
(1): Linear(in_features=16384, out_features=256, bias=True)
(2): ReLU()
(3): Dropout(p=0.3, inplace=False)
(4): Linear(in_features=256, out_features=3, bias=True)
)
)
Variables Terminal
7:47 PM Python 3


PES2UG23CS134_Week14_Lab.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[7]
5m
# 3. Calculate the loss (using criterion)
loss = criterion(outputs, labels)
# 4. Perform a backward pass (loss.backward())
loss.backward()
# 5. Update the weights (optimizer.step())
optimizer.step()
total_loss += loss.item()
print(f"Epoch {epoch+1}/{EPOCHS}, Loss = {total_loss/len(train_loader):.4f}")
print("Training complete!")
Epoch 1/10, Loss = 0.6251
Epoch 2/10, Loss = 0.1913
Epoch 3/10, Loss = 0.0883
Epoch 4/10, Loss = 0.0568
Epoch 5/10, Loss = 0.0229
Epoch 6/10, Loss = 0.0139
Epoch 7/10, Loss = 0.0141
Epoch 8/10, Loss = 0.0077
Epoch 9/10, Loss = 0.0091
Epoch 10/10, Loss = 0.0261
Training complete!
Step 6: Evaluate the Model

PES2UG23CS134_Week14_Lab.ipynb

File Edit View Insert Runtime Tools Help

ommands + Code + Text Run all

```
Epoch 9/10, Loss = 0.0091
Epoch 10/10, Loss = 0.0261
Training complete!
```

Step 6: Evaluate the Model

Test the model's accuracy on the unseen test set.

[8] ✓ 5s

```
model.eval() # Set the model to evaluation mode
correct = 0
total = 0

# TODO: Use torch.no_grad()
# We don't need to calculate gradients during evaluation
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)

        # TODO: Get model predictions
        # 1. Get the raw model outputs (logits)
        outputs = model(images) # <-- Replace this

        # 2. Get the predicted class (the one with the highest score)
        #    Hint: use torch.max(outputs, 1)
        _, predicted = torch.max(outputs.data, 1) # <-- Replace this

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Test Accuracy: {100 * correct / total:.2f}%")
```

```
Test Accuracy: 96.80%
```

Step 7: Test on a Single Image